

Fundación Fulgor
PROCOM

Informe

Integración DSP y uP

Autor: Alfici, Facundo Ezequiel

Documento: 44012327

Docentes: Ariel Pola, Santiago Garaventta Pascual y Agustín Pesce

6 de septiembre de 2026

Índice

1. Introducción	3
2. Desarrollo	4
2.1. Módulo FPGA	4
2.2. Módulo RF (register_file)	4
2.3. Módulo BRAM (mem_log)	5
2.4. Módulo DSP (dsp_core)	6
2.5. Firmware del MicroBlaze	6
2.6. Python Script	7
2.7. Resultados obtenidos	8
3. Conclusiones	12

1. Introducción

En este informe, se detallará el funcionamiento de cada bloque del sistema implementado, haciendo énfasis en la integración entre el microcontrolador y el módulo de procesamiento digital de señales (DSP). También se presentarán los resultados obtenidos durante las pruebas realizadas, así como las conclusiones a las que se llegó tras la implementación del proyecto.

En primera medida, se detallará la arquitectura del sistema mediante la siguiente figura, que refleja la forma en la cuál los 4 bloques principales conviven entre sí para lograr el objetivo final.

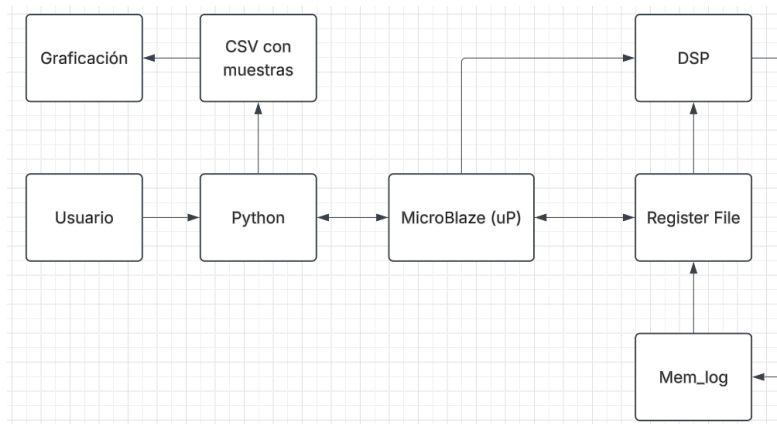


Figura 1.0.1: Diagrama en bloques de la comunicación de la implementación.

Los bloques principales que componen el sistema son los siguientes:

- **MicroBlaze:** Este bloque se encarga del control de la información mediante el firmware cargado.
- **Procesamiento digital de señales (DSP):** Este bloque es el encargado de realizar el procesamiento de la información generada por la PRBS9, aplicada a un filtro polifásico RC y analizada mediante un ber counter.
- **Mem_log:** Este bloque está compuesto por una BRAM, permite almacenar muestras provenientes de la salida del filtro.
- **Register File:** Finalmente, este bloque se encarga de manejar los comandos enviados por el microprocesador y la información proveniente del DSP de la BRAM.

2. Desarrollo

Se dividirá el desarrollo del informe en 6 secciones, 5 de ellas explicando cada bloque funcional por separado, dejando la última sección para la muestra de resultados obtenidos.

2.1. Módulo FPGA

Este bloque se encarga de la integración a nivel global, instancia los 4 bloques funcionales del sistema, siguiendo el esquema provisto en la figura 1.0.1. Además, contiene la codificación del bloque MicroGPIO, el cual a su vez contiene el bloque del **MicroBlaze**, el **UART** y el **AXI GPIO**, exportados mediante el IP Block Design.

Expone hacia el resto del diseño el puerto de salida GPO0 de 32 bit, el puerto de entrada GPIO0 y la señal locked del clocking wizard, que sirve como reset general del sistema. También posee un clocking wizard encargado de generar el clock de 100MHz que se usa a nivel global.

Como consigna se tuvo que realizar un cambio respecto a la implementación del laboratorio N°5, donde se utilizaban los leds como lectura del estado de los switches. En este caso, se utilizan `out_leds[0]` para la señal locked, `out_leds[1]`, `[2]`, `[3]` para reflejar el estado del reset, la habilitación de la transmisión y la habilitación de la recepción. De igual manera, se utilizaron los leds RGB para describir el estado de la memoria y de la fase seleccionada, siendo RGB0 para la memoria y RGB1 para la fase y la indicación de ber nula por fase.

2.2. Módulo RF (register_file)

Este bloque contiene un banco de registros de propósito general, que se utilizan dentro del diseño para controlar las variables, acciones y flags del sistema de comunicaciones que existe entre los módulos. Implementa el almacenamiento de los comandos como se muestra en la siguiente figura.

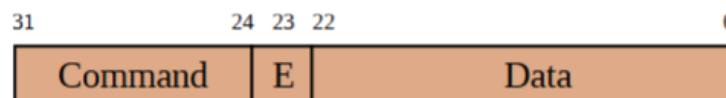


Figura 2.2.1: Particionado de bits de salida del GPIO.

El proceso de escritura lo lleva a cabo el procesador, el cual escribe 3 veces el puerto de salida con el mismo comando e información, cambiando solamente el bit

de habilitación. Este bit de habilitación tiene la importante tarea de avisarle al registro si el dato recibido es confiable o no.

La primera vez que se escribe la información en el puerto de salida del procesador, el bit de enable se encuentra en 0. El registro lo recibe y espera una nueva muestra, la cuál debería venir con el bit de enable en 1. Finalmente, la última muestra se recibe nuevamente con el bit de enable en 0. De esta manera se puede hacer la verificación del comando, dado que si hay cualquier tipo de ruido en la comunicación la señal podría llegar tergiversada.

Los tipos de comandos dependerán de lo que se desee realizar por el usuario, por lo que queda a libre interpretación del diseñador, para este trabajo se implementaron los siguientes comandos:

- **o_rst:** Envía un reset generalizado.
- **o_enb_tx:** Habilitación de la transmisión.
- **o_enb_rx:** Habilitación de la recepción.
- **o_run_log:** Activar escritura de la memoria.
- **o_read_log:** Leer memoria.
- **o_addr_log_to_mem:** Dirección de memoria específica.
- **o_rd_sel:** Seleccionar el código a leer.
- **o_ber_rst:** Reset de contadores de BER. (Implementado por un error en el diseño provisto en el Lab.Nº4, donde la BER solo se reseteaba al cambiar de fase.)

2.3. Módulo BRAM (mem_log)

Representa la implementación de la BRAM, con una capacidad de 32K x 32b. La escritura de la memoria se activa mediante el flanco ascendente del comando **o_run_log**, de tal manera que la memoria llena sus casillas a una velocidad de clock. Una vez la memoria se llena, se detiene la escritura y se activa la flag **i_mem_full**, que como su nombre indica, avisa que la memoria está llena y lista para la lectura.

Por el contrario, el proceso de lectura usa **o_addr_log_to_mem** como dirección y **o_read_log** como comando de habilitación. Una vez llega el comando de habilitación, el dato solicitado se carga y se envía por medio de **i_data_log_from_mem**.

Algo importante a tener en cuenta es que la lectura se bloquea al escribir para evitar errores graves de pérdida de información, siendo este bloqueo controlado por

el firmware del MicroBlaze, donde deshabilita `o_read_log` antes de que la memoria envíe datos.

2.4. Módulo DSP (`dsp_core`)

Finalizando con los módulos de verilog, no hay mucho de este módulo que no se haya comentado ya en anteriores laboratorios, consiste de los siguientes submódulos:

- *PRBS9*: Genera una secuencia pseudoaleatoria de 9 bits. Tiene 2 instancias, una para los datos de entrada del upscaling y otro para la generación de la referencia para el contador de BER.
- *control*: Encargado de decodificar las señales provenientes del Register File, como ser `en_tx`, `en_rx` y `phase_sw`. A su vez, resetea el contador de BER al detectar un cambio de fase.
- *freq_divider*: Genera la base de tiempo que utiliza el banco de coeficientes del filtro polifásico para aplicar el filtrado de los bits enviados por la PRBS9.
- *rc_tx*: Contiene el filtro polifásico de coseno realzado (RC) junto con sus coeficientes. Este módulo recibe el selector de fase proveniente del *freq_divider* que funciona como selector, de tal manera entrega cuatro muestras interpoladas por símbolo. Finalmente, se aplica una sumatoria de cada tap según el bit almacenado en el registro de desplazamiento.
- *decimator*: Como su mismo nombre lo indica, se encarga de tomar una muestra por símbolo según la fase seleccionada, también actúa como slicer para quedarse con el bit de signo.
- *ber_counter*: Por medio de la PRBS9, compara los bits enviados desde el *decimator* con los generados por cada semilla. Dado que se debe hacer la comparación correcta, la referencia tiene que estar alineada con el retardo de la cadena total. También es destacable la implementación de una compensación para el cambio de fase.

2.5. Firmware del MicroBlaze

Escrito en **lenguaje C**, se encarga de hacer de intérprete entre las tramas enviadas por UART y el GPIO. El bucle principal de funcionamiento espera un header válido de iniciación y uno válido de finalización, para luego enviar los datos según el device seleccionado. Es importante destacar que en caso de no obtener una igualdad entre el header esperado ($101+L/S+Size$) y el recibido se

realizará el descarte de la trama completa. Lo mismo ocurre para el trailer esperado ($010+L/S+Size$) y el recibido.

Para la lectura de trama, el bit de Size y el tamaño de la información en caso de ser una trama corta deben coincidir tanto en el **Header** como en el **Trailer**.

También implementa el protocolo de escritura de la figura 2.2.1, donde arma tramas de 32bits con el comando en la parte alta y el dato en la parte baja, escribiendo 3 veces el puerto GPIO para que se cumpla la verificación del bit de enable (Strobing).

El código pretende abarcar 3 dispositivos, los cuales se detallan a continuación:

- **DEV_RF_WRITE:** Escritura del Register File. Recibe 4 bytes; El comando y el dato. Luego llama al protocolo de strobeo.
- **DEV_RF_READ:** Lectura del Register File. Recibe un byte con el código de lectura.
- **DEV_READ_SW:** Lectura de los 4 bits de los switches.

2.6. Python Script

Este script representa la interfaz de usuario, donde se escriben los comandos para controlar el sistema. Consta de tres capas:

1. **Capa de trama:** Tiene una función encargada de armar la trama y seleccionar el tipo de formato según el tamaño de la información, es decir, distingue entre trama corta y trama larga. También consta de otra función que hace lo contrario para los casos donde se necesite recibir datos.
2. **Capa de registros:** Construye la lectura y escritura del register file, de tal manera que luego se puede leer el status del registro, leer el dato proporcionado por los contadores de BER o leer la salida del filtro Tx.
3. **Capa de comandos:** Es la consola que maneja los comandos que permiten exponer cada registro del register file como un comando junto con otras tres operaciones compuestas, también encontramos comandos tales como *ber_sweep*, que mide la BER de las cuatro fases, y *dumplog*, que descarga cierta cantidad de muestras a un archivo .csv para luego graficarlas mediante otro script.

El script que genera las gráficas del *dumplog* permite visualizar la salida del filtro para cada una de las fases, superponiéndolas con las muestras que le toca a cada fase teniendo en cuenta el decimador para la fase seleccionada. Por otro lado, genera un diagrama de ojo para fase y cuadratura.

Los comandos que posee la interfaz de usuario se listan a continuación:

- `rst <0/1>`: Reset del DSP.
- `entx <0/1>| enrxd <0/1>`: Habilitación de transmisión y recepción.
- `phase <0-3>`: Selección de fase.
- `sw`: Lectura de llaves.
- `status`: Lectura de los registros del RF.
- `ber`: Lectura de los contadores de muestras y errores para la fase seleccionada.
- `berrst`: Reset de los contadores de la fase actual.
- `bersweep`: Ber de las cuatro fases.
- `runlog`: Habilitación de carga de memoria.
- `memfull`: Read de flag de memoria llena.
- `addr <n>`: Fija una dirección de memoria
- `readword <n>`: Lee una dirección de memoria.
- `dumplog <n>[archive]`: Guarda n muestras de la memoria a un archivo .csv
- `buildid`: Permite ver la versión actual del bitstream.
- `help`: Relanzar lista de comandos.
- `exit`: Salir de la comunicación.

2.7. Resultados obtenidos

Comenzando con este apartado, se desea llevar a cabo el proceso de comunicación completo mediante los comandos diseñados. Teniendo esto en cuenta, se escribieron los siguientes comandos;

```
<< buildid
>> build id = 0xC0FFEE04
<< rst 0
<< entx 1
<< enrxd 1
<< phase 2
<< addr 4660
<< status
>> 0x00091A16 {'rst': 0, 'enb_tx': 1, 'enb_rx': 1, 'phase': 2, 'run_log': 0, 'read_log': 0, 'addr_log': 4660}
<<
```

Figura 2.7.1: Comandos para inicializar la comunicación

En la figura 2.7.1 se pueden ver el funcionamiento de los comandos de inicialización de la comunicación, como ser la habilitación de la transmisión y recepción, la selección de fase o el estado del reset. Los cuales se muestran mediante **status**. Consiguientemente, se tiene la seguidilla de comandos que permiten ver la BER para una fase en específico.

```
<< phase 0
<< entx 1
<< enrx 1
<< rst 1
<< rst 0
<< ber
>> I: samp=38990146 err=0 BER=0.000e+00
>> Q: samp=39790590 err=0 BER=0.000e+00
<< ber
>> I: samp=80987401 err=0 BER=0.000e+00
>> Q: samp=81787645 err=0 BER=0.000e+00
<< |
```

Figura 2.7.2: BER para la fase 0.

Lo mismo se puede plantear para todas las fases, obteniendo las siguientes lecturas.

```
<< bersweep
fase  samp_I      err_I      BER_I      BER_Q      esperado
0      7631928      0          0.0000     0.0000     0.00
1      7633241      0          0.0000     0.0000     0.00
2      7634292      1912311    0.2505     0.2505     0.25
3      7633871      0          0.0000     0.0000     0.00
<< |
```

Figura 2.7.3: BER de todas las fases.

En lo que respecta a la RAM, las siguientes imágenes muestran la escritura y lectura.

```

<< phase 0
<< entx 1
<< enrx 1
<< rst 1
<< rst 0
<< memfull
>> mem_full = 0
<< runlog
<< memfull
>> mem_full = 1
<< |

```

Figura 2.7.4: Escritura de la RAM.

```

<< readword 0
>> word=0xFFA90057 sample_I=-87 sample_Q=87
<< readword 1
>> word=0xFFA00048 sample_I=-96 sample_Q=72
<< readword 2
>> word=0xFF980016 sample_I=-104 sample_Q=22
<< readword 100
>> word=0x00570057 sample_I=87 sample_Q=87
<< readword 1000
>> word=0xFFA90057 sample_I=-87 sample_Q=87
<< |

```

Figura 2.7.5: Lectura de la RAM.

La carga de datos al archivo .csv permite generar los siguientes gráficos, tomando como referencia a la fase 0 a la hora de la carga.

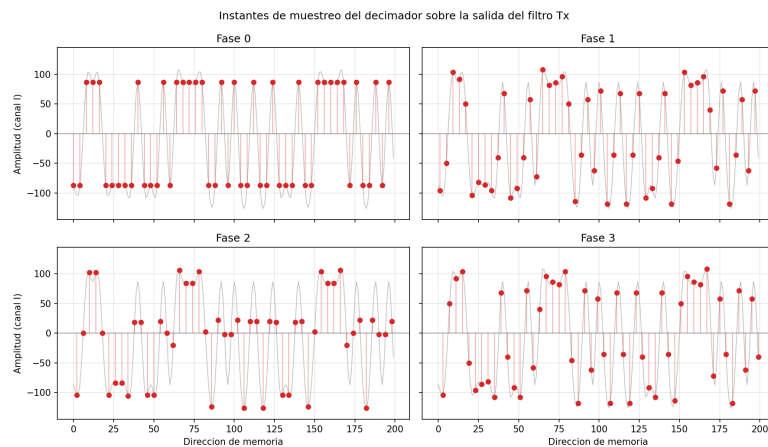


Figura 2.7.6: Salida filtro Tx para todas las fases.

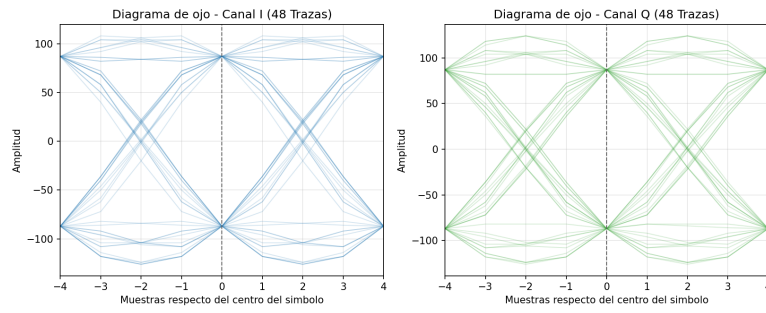


Figura 2.7.7: Diagrama de ojos de fase y cuadratura.

Por último, el comando de lectura de los switches se puede ver a continuación.

```
<< SW
>> switches = 0111
```

Figura 2.7.8: Lectura de switches.

3. Conclusiones

Finalizando este informe, se concluye que el sistema funciona debidamente, y comparando con los resultados obtenidos de la salida del filtro y el diagrama de ojos, con los esperados según las simulaciones realizadas en punto fijo de laboratorios anteriores, la salida obtenida es de buena calidad. De igual manera, la implementación de la memoria y de la interfaz de usuario permite un espacio más amigable para el control del sistema, evitando el uso de VIO e ILA para este caso concreto.

Las mayores dificultades se obtuvieron con Vitis, donde la imposibilidad de switchear el archivo .xsa (Exportación del Hardware desde Vivado), generó la necesidad de crear un comando que permita ver la versión actual del bitstream y evitar así interpretaciones erróneas.